

Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods

Chief Strategist J

June 12, 2026

Contents

I	Foundation Training Methods	3
1	Pretraining	3
1.1	Mathematical Foundation	3
1.2	Architecture Flow Diagram	3
1.3	State Transition Table	3
2	Continued Pretraining (CPT)	4
2.1	Mathematical Foundation	4
2.2	Architecture Flow Diagram	5
2.3	State Transition Table	5
3	Domain Adaptive Pretraining (DAPT)	6
3.1	Mathematical Foundation	6
3.2	Architecture Flow Diagram	6
3.3	State Transition Table	6
4	Task Adaptive Pretraining (TAPT)	7
4.1	Mathematical Foundation	7
4.2	Architecture Flow Diagram	7
4.3	State Transition Table	7
II	Supervised Fine-Tuning Methods	8
5	Supervised Fine-Tuning (SFT)	8
5.1	Mathematical Foundation	8
5.2	Architecture Flow Diagram	8
5.3	State Transition Table	8
6	Instruction Tuning	9
6.1	Mathematical Foundation	9
6.2	Architecture Flow Diagram	9
6.3	State Transition Table	9
7	Multi-Task Training	10
7.1	Mathematical Foundation	10
7.2	Architecture Flow Diagram	10
7.3	State Transition Table	10
8	Chain-of-Thought SFT	11
8.1	Mathematical Foundation	11
8.2	Architecture Flow Diagram	11
8.3	State Transition Table	11

9	Step-by-Step SFT	12
9.1	Mathematical Foundation	12
9.2	Architecture Flow Diagram	12
9.3	State Transition Table	12
10	Process Supervision	13
10.1	Mathematical Foundation	13
10.2	Architecture Flow Diagram	13
10.3	State Transition Table	13
III	Parameter-Efficient Fine-Tuning (PEFT)	14
11	Low-Rank Adaptation (LoRA)	14
11.1	Mathematical Foundation	14
11.2	Architecture Flow Diagram	14
11.3	State Transition Table	14
12	Quantized LoRA (QLoRA)	15
12.1	Mathematical Foundation	15
12.2	Architecture Flow Diagram	15
12.3	State Transition Table	15
13	Adaptive LoRA (AdaLoRA)	16
13.1	Mathematical Foundation	16
13.2	Architecture Flow Diagram	16
13.3	State Transition Table	16
14	Weight-Decomposed LoRA (DoRA)	17
14.1	Mathematical Foundation	17
14.2	Architecture Flow Diagram	17
14.3	State Transition Table	17
15	IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations)	18
15.1	Mathematical Foundation	18
15.2	Architecture Flow Diagram	18
15.3	State Transition Table	18
16	Adapter Tuning	19
16.1	Mathematical Foundation	19
16.2	Architecture Flow Diagram	19
16.3	State Transition Table	19
17	Prefix Tuning	20
17.1	Mathematical Foundation	20
17.2	Architecture Flow Diagram	20
17.3	State Transition Table	20
18	Prompt Tuning	21
18.1	Mathematical Foundation	21
18.2	Architecture Flow Diagram	21
18.3	State Transition Table	21
19	P-Tuning v2	22
19.1	Mathematical Foundation	22
19.2	Architecture Flow Diagram	22
19.3	State Transition Table	22

Part I

Foundation Training Methods

Pretraining

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t \mid x_{<t}) \quad (1)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (2)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (3)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \quad (4)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t \mid x_{<t}) \quad (5)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (6)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (7)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (8)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (9)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (10)$$

Architecture Flow Diagram

State Transition Table

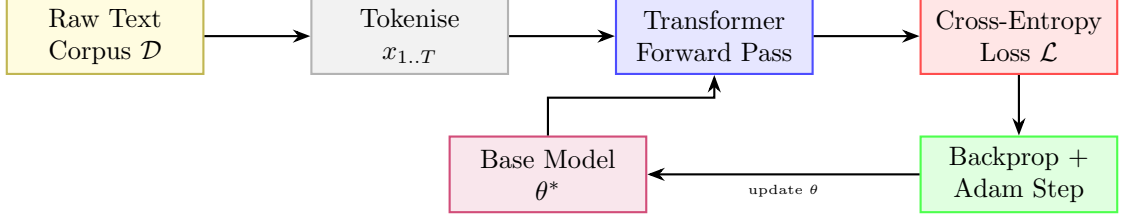


Figure 1: Pretraining pipeline: corpus sampling, forward pass, loss computation, and weight update loop.

Property	Before	After
Parameter source	Random init $\theta_0 \sim \mathcal{N}(0, \sigma^2)$	Converged weights θ^*
Training corpus	Not consumed	Trillions of tokens processed
Perplexity	Very high ($> 10^3$)	Low (< 20 on held-out data)
World knowledge	None	Broad language & factual knowledge
requires_grad	True (all params)	True (all params)

Table 1: State properties before and after pretraining.

Continued Pretraining (CPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t | x_{<t}) \quad (11)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (12)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (13)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v | x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \quad (14)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t | x_{<t}) \quad (15)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (16)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (17)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (18)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (19)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (20)$$

Architecture Flow Diagram

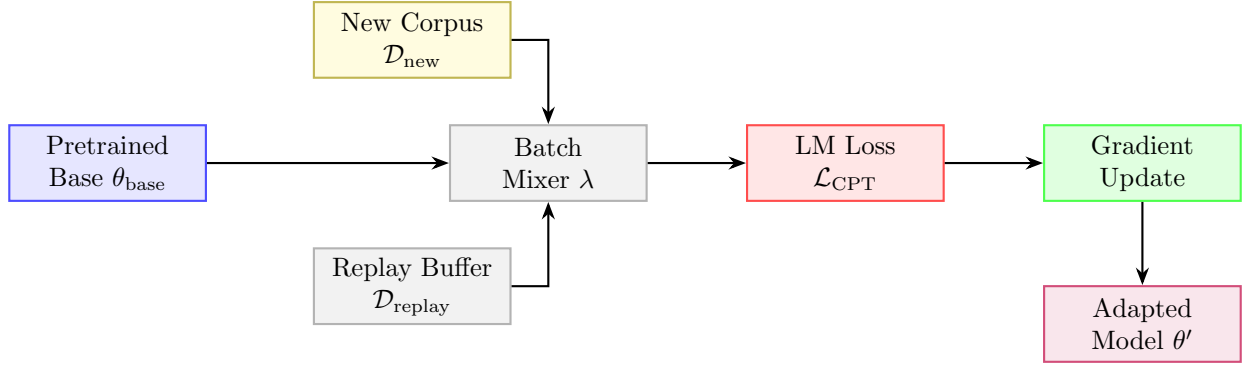


Figure 2: CPT mixes new corpus and replay buffer batches to update a pretrained checkpoint.

State Transition Table

Property	Before	After
Starting checkpoint	Pretrained θ_{base}	$\theta' = \theta_{\text{base}} + \Delta\theta$
New domain knowledge	Absent	Incorporated
Original knowledge	Retained	Partially retained via replay
Learning rate schedule	Finished	Warm-restart applied
Perplexity on new domain	High	Significantly reduced

Table 2: State properties before and after continued pretraining.

Domain Adaptive Pretraining (DAPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t | x_{<t}) \quad (21)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (22)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (23)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v | x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \quad (24)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t | x_{<t}) \quad (25)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (26)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (27)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (28)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (29)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (30)$$

Architecture Flow Diagram

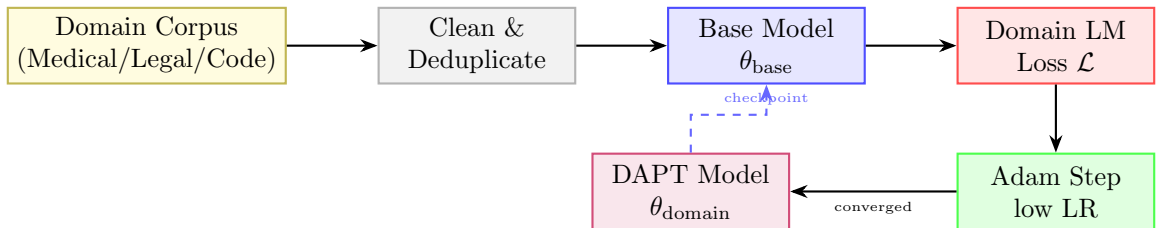


Figure 3: DAPT pipeline: domain corpus is cleaned, fed into the base model, and trained with standard LM loss.

State Transition Table

Property	Before	After
Domain vocabulary familiarity	Low	High
Domain perplexity	High ($\rho \gg 1$)	Low (near 1)
General language ability	Strong	Marginally reduced
Downstream task (same domain)	Weak	Significantly stronger
Training data type	General web text	Domain-specific unlabelled

Table 3: State properties before and after domain adaptive pretraining.

Task Adaptive Pretraining (TAPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t \mid x_{<t}) \quad (31)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (32)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (33)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \quad (34)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t \mid x_{<t}) \quad (35)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (36)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (37)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (38)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (39)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (40)$$

Architecture Flow Diagram

State Transition Table

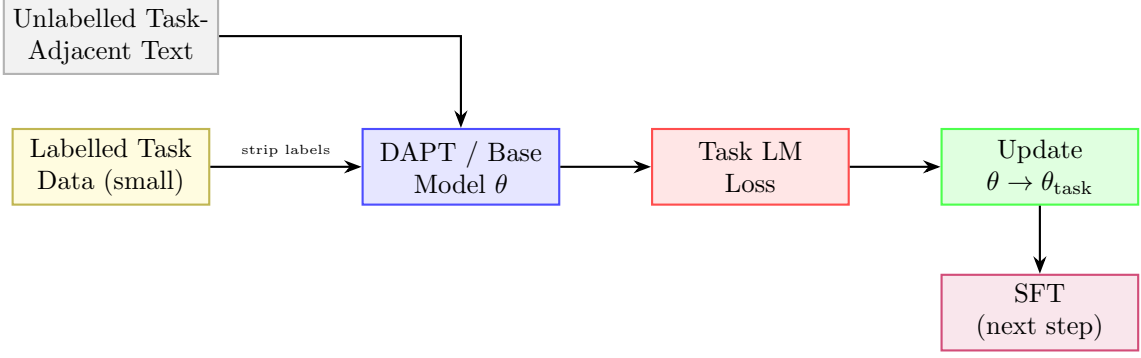


Figure 4: TAPT uses task-adjacent unlabelled text to bridge base model and supervised fine-tuning.

Property	Before	After
Task vocabulary alignment	Partial	Strong
Corpus size	Millions of docs	Thousands of docs
Training epochs	1–3	10–100 (small corpus)
Downstream label efficiency	Requires more labels	Requires fewer labels
Transfer to other tasks	Good	Reduced (narrow focus)

Table 4: State properties before and after task adaptive pretraining.

Part II

Supervised Fine-Tuning Methods

Supervised Fine-Tuning (SFT)

Mathematical Foundation

SFT trains the model on structured prompt-response pairs $(x, y) \sim \mathcal{D}_{\text{SFT}}$ using teacher forcing. Let $s = [x; y]$ be the concatenated token sequence of length $M + U$, where $x = (s_1, \dots, s_M)$ is the prompt and $y = (s_{M+1}, \dots, s_{M+U})$ is the response. The key math lies in applying a target label mask $L \in \{-100, \mathcal{V}\}^{M+U}$ defined as:

$$L_t = \begin{cases} -100 & \text{if } t \leq M \\ s_t & \text{if } t > M \end{cases} \quad (41)$$

The cross-entropy loss function ignores targets set to -100 , directing gradient updates exclusively to the response generation tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{U} \sum_{t=M+1}^{M+U} \log P_{\theta}(s_t \mid s_{<t}) \quad (42)$$

This loss ignores the prompt distribution $P(x)$, preventing the model from degrading its output generation capability to match arbitrary prompts. The gradients only backpropagate through response tokens:

$$\nabla_{\theta} \mathcal{L}_{\text{SFT}} = -\frac{1}{U} \sum_{t=M+1}^{M+U} \nabla_{\theta} \log P_{\theta}(s_t \mid s_{<t}) \quad (43)$$

Architecture Flow Diagram

State Transition Table

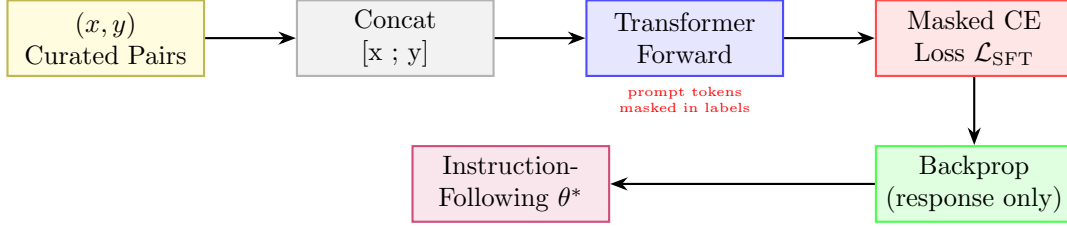


Figure 5: SFT pipeline with prompt-masked loss: gradients flow only through response tokens.

Property	Before	After
Behaviour	Completion only	Instruction following
Loss region	N/A	Response tokens only
Label masking	None	Prompt masked to -100
Data format	Raw text	(instruction, response) pairs
Output quality	Random completion	Task-aligned responses

Table 5: State properties before and after supervised fine-tuning.

Instruction Tuning

Mathematical Foundation

Instruction Tuning exposes the model to multi-turn conversational patterns. A prompt template $T(I, x)$ converts an instruction I and context x into structured sequences. Turn boundaries are defined using chat-formatting systems (like ChatML). A single sequence layout is:

$$S = \langle \text{user} \rangle I \parallel x \langle \text{assistant} \rangle y \langle \text{eos} \rangle \quad (44)$$

Let $\mathcal{D}_{\text{inst}}$ consist of instructions spanning hundreds of tasks. The model parameters θ are optimized on the joint multi-task dataset:

$$\mathcal{L}_{\text{IT}}(\theta) = -\mathbb{E}_{(I, x, y) \sim \mathcal{D}_{\text{inst}}} \sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid T(I, x), y_{<t}) \quad (45)$$

We mask all prompt-side tokens, including conversational format tags (like $\langle \text{user} \rangle$ and instruction text), ensuring parameters are trained only on synthesizing assistant responses:

$$\mathcal{L}_{\text{IT}}(\theta) = -\frac{1}{|y|} \sum_{i=1}^{|T(I, x)| + |y|} M_i \log P_{\theta}(S_i \mid S_{<i}), \quad M_i = \mathbb{I}(i > |T(I, x)|) \quad (46)$$

Architecture Flow Diagram

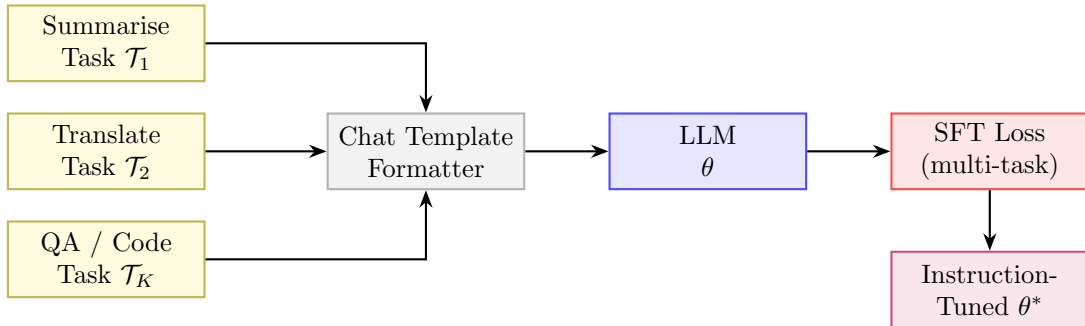


Figure 6: Instruction tuning over diverse task types funnelled through a unified chat template.

State Transition Table

Property	Before	After
Zero-shot generalisation	Poor	Strong across task types
Task diversity in training	N/A	K heterogeneous tasks
Input format	Free-form text	Structured chat template
Prompt sensitivity	High	Reduced
Held-out task performance	Weak	Strong (FLAN, InstructGPT)

Table 6: State properties before and after instruction tuning.

Multi-Task Training

Mathematical Foundation

Multi-Task Training aggregates losses from K distinct tasks. A simple summation can result in tasks with large gradient scales dominating the parameter updates. To resolve this, we use the GradNorm algorithm. Let W be the shared parameter weights (e.g., the last transformer block weights). We define the gradient norm for task k at training step t as:

$$G_k(t) = \|\nabla_W(w_k(t)\mathcal{L}_k(t))\|_2 \quad (47)$$

Let $\bar{G}(t) = \frac{1}{K} \sum_k G_k(t)$ be the average gradient norm. The target gradient norm for task k balances task difficulties:

$$\hat{G}_k(t) = \bar{G}(t) \times \left(\frac{\tilde{\mathcal{L}}_k(t)}{\frac{1}{K} \sum_j \tilde{\mathcal{L}}_j(t)} \right)^\alpha \quad (48)$$

where $\tilde{\mathcal{L}}_k(t) = \mathcal{L}_k(t)/\mathcal{L}_k(0)$ is the task loss ratio indicating task training progress, and α is a hyperparameter managing structural gradient balancing. The task weights $w_k(t)$ are updated by minimizing the L1 distance between $G_k(t)$ and $\hat{G}_k(t)$:

$$\mathcal{L}_{\text{grad}}(w_1, \dots, w_K; t) = \sum_{k=1}^K |G_k(t) - \hat{G}_k(t)| \quad (49)$$

The final joint parameter objective is:

$$\mathcal{L}_{\text{MT}}(\theta) = \sum_{k=1}^K w_k \mathcal{L}_k(\theta) \quad (50)$$

Architecture Flow Diagram

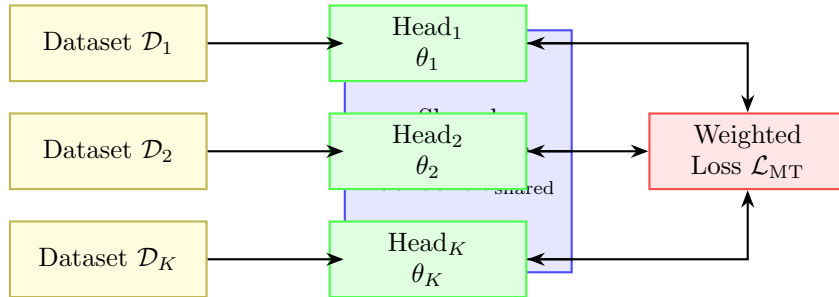


Figure 7: Multi-task training: each dataset feeds task-specific heads on a shared backbone.

State Transition Table

Property	Before	After
Backbone specialisation	Single task	K tasks jointly
Task heads	1	K heads
Generalisation	Narrow	Broad
Data efficiency	Per-task only	Cross-task transfer
Gradient conflicts	N/A	Managed via λ_k

Table 7: State properties before and after multi-task training.

Chain-of-Thought SFT

Mathematical Foundation

CoT SFT trains models to output intermediate reasoning steps before generating the final answer. Let x be the prompt, $c = (c_1, \dots, c_M)$ be the reasoning chain tokens, and $y = (y_1, \dots, y_U)$ be the final answer tokens. The joint probability factorization is:

$$P(c, y \mid x) = \prod_{i=1}^M P_{\theta}(c_i \mid x, c_{<i}) \prod_{j=1}^U P_{\theta}(y_j \mid x, c, y_{<j}) \quad (51)$$

The model is trained by minimizing the joint loss:

$$\mathcal{L}_{\text{CoT}}(\theta) = -\beta \sum_{i=1}^M \log P_{\theta}(c_i \mid x, c_{<i}) - (1 - \beta) \sum_{j=1}^U \log P_{\theta}(y_j \mid x, c, y_{<j}) \quad (52)$$

During autoregressive generation, we sample tokens using Temperature-scaled softmax and Nucleus (top- p) filtering. The logits z_t are modified as:

$$\tilde{z}_{t,i} = \frac{z_{t,i}}{T} \quad (53)$$

where $T > 0$ is the temperature. The top- p vocabulary subset $\mathcal{V}^{(p)}$ at step t is the smallest set satisfying:

$$\sum_{v \in \mathcal{V}^{(p)}} P(v \mid x, s_{<t}) \geq p \quad (54)$$

Tokens outside $\mathcal{V}^{(p)}$ are assigned probability 0 before final selection.

Architecture Flow Diagram

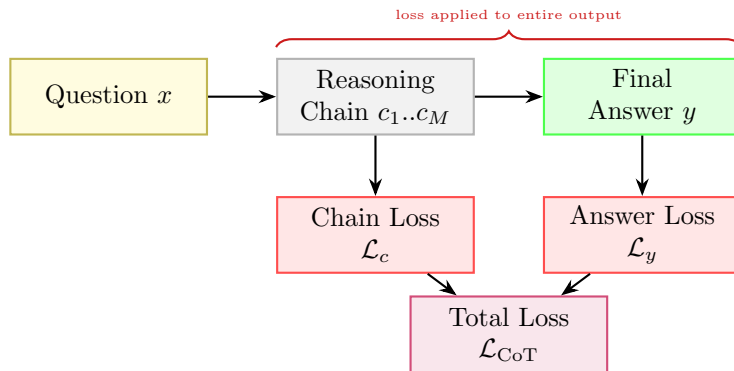


Figure 8: CoT SFT: loss is computed over both the reasoning chain and the final answer.

State Transition Table

Property	Before	After
Output format	Direct answer	Reasoning chain + answer
Training signal per example	$ y $ tokens	$ c + y $ tokens
Multi-step reasoning	Weak	Strong
Data annotation cost	Low	High (chain annotation)
Accuracy on complex tasks	Low	Substantially improved

Table 8: State properties before and after Chain-of-Thought SFT.

Step-by-Step SFT

Mathematical Foundation

Step-by-Step SFT formats the reasoning chain into discrete steps separated by a boundary token $T_{\text{step}} = [\text{STEP}]$. Let the response contain S steps: $s = (s_1, T_{\text{step}}, s_2, T_{\text{step}}, \dots, s_S, T_{\text{step}})$. The step-level loss is:

$$\mathcal{L}_{\text{SBS}}(\theta) = - \sum_{k=1}^S \sum_{t=1}^{|s_k|} \log P_{\theta}(s_{k,t} \mid x, s_{<k}, s_{k,<t}) - \sum_{k=1}^S \log P_{\theta}(T_{\text{step}} \mid x, s_{\leq k}) \quad (55)$$

We define the average step perplexity PPL_k as:

$$\text{PPL}_k = \exp \left(- \frac{1}{|s_k|} \sum_{t=1}^{|s_k|} \log P_{\theta}(s_{k,t} \mid x, s_{<k}, s_{k,<t}) \right) \quad (56)$$

Steps with high perplexity ($\text{PPL}_k > \tau$, where τ is a threshold) indicate low model confidence and can be flagged for step-level revision during generation. The model is trained to minimize the combined cross-entropy loss over all steps.

Architecture Flow Diagram

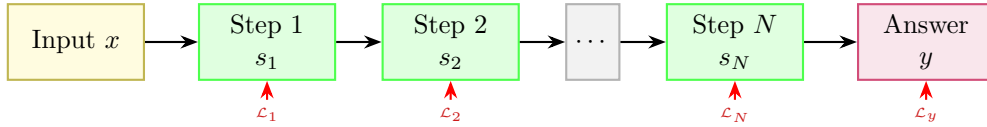


Figure 9: Step-by-Step SFT: each numbered step and the final answer contribute to the total loss.

State Transition Table

Property	Before	After
Solution structure	Unstructured	Numbered procedural steps
Step delimiter tokens	Not in vocabulary	Learned as first-class tokens
Interpretability	Low	Step-level traceability
Annotation format	Raw answer	$(x, [s_1, \dots, s_N], y)$
Task decomposition ability	Implicit	Explicit

Table 9: State properties before and after Step-by-Step SFT.

Process Supervision

Mathematical Foundation

Process Supervision trains a Process Reward Model (PRM) using step-level binary correctness labels. Let x be the problem description, and $s_1, \dots, s_S$ be the generated steps. Human or model-based annotations assign correctness labels $r_k \in \{0, 1\}$ for each step s_k . The PRM model outputs correctness probabilities by applying a logistic sigmoid function to the step boundary token representation:

$$\hat{r}_k = \sigma(\text{PRM}_\phi(h_{t_k})) = \frac{1}{1 + \exp(-\text{PRM}_\phi(h_{t_k}))} \quad (57)$$

where h_{t_k} is the transformer hidden output at index t_k representing the step boundary token. The PRM parameters ϕ are optimized using binary cross entropy:

$$\mathcal{L}_{\text{PRM}}(\phi) = - \sum_{k=1}^S [r_k \log \hat{r}_k + (1 - r_k) \log(1 - \hat{r}_k)] \quad (58)$$

During inference, candidate sequences are generated. The total sequence correctness score is calculated as the product of step correctness probabilities:

$$\text{Score}(s_{\leq S}) = \prod_{k=1}^S \hat{r}_k \quad (59)$$

This score is used to rank candidate rollouts in Best-of- N sampling or guide selection during Monte Carlo Tree Search (MCTS).

Architecture Flow Diagram

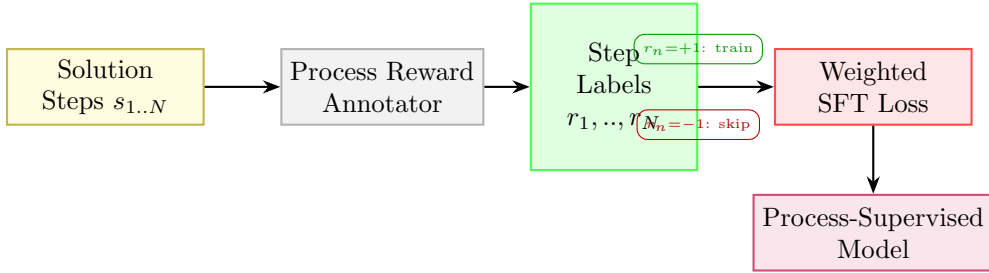


Figure 10: Process supervision: a step annotator labels each reasoning step; only correct steps contribute to the SFT loss.

State Transition Table

Property	Before	After
Supervision granularity	Final answer only	Per-step labels
Error signal	Outcome-level	Step-level
Error localisation	None	Identifies which step failed
Annotation cost	Low	High (step-level labelling)
Reasoning quality	Outcome-driven	Step-correctness driven

Table 10: State properties before and after process supervision.

Part III

Parameter-Efficient Fine-Tuning (PEFT)

Low-Rank Adaptation (LoRA)

Mathematical Foundation

LoRA represents the weight update matrix $\Delta W \in \mathbb{R}^{k \times d}$ for a linear projection layer as the product of two low-rank matrices $B \in \mathbb{R}^{k \times r}$ and $A \in \mathbb{R}^{r \times d}$, where the rank satisfies $r \ll \min(d, k)$:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x \quad (60)$$

where α is a constant scaling parameter. The base weight matrix $W_0 \in \mathbb{R}^{k \times d}$ is frozen; only B and A are updated. Under a loss $\mathcal{L}$, the gradient propagation rules using the chain rule are derived as follows: Let $z = Ax \in \mathbb{R}^r$ be the intermediate bottleneck activation. Then $h = W_0 x + \frac{\alpha}{r} B z$. The gradient with respect to output activation is $\nabla_h \mathcal{L} \in \mathbb{R}^{k \times 1}$ (represented as a column vector). The gradient with respect to the intermediate bottleneck representation z :

$$\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} \in \mathbb{R}^{r \times 1} \quad (61)$$

The gradient with respect to the matrix B :

$$\nabla_B \mathcal{L} = \frac{\alpha}{r} \nabla_h \mathcal{L} z^\top = \frac{\alpha}{r} \nabla_h \mathcal{L} x^\top A^\top \in \mathbb{R}^{k \times r} \quad (62)$$

The gradient with respect to the matrix A :

$$\nabla_A \mathcal{L} = \nabla_z \mathcal{L} x^\top = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} x^\top \in \mathbb{R}^{r \times d} \quad (63)$$

To eliminate latency at deployment, the weights are merged directly into the base model:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} B A \quad (64)$$

Architecture Flow Diagram

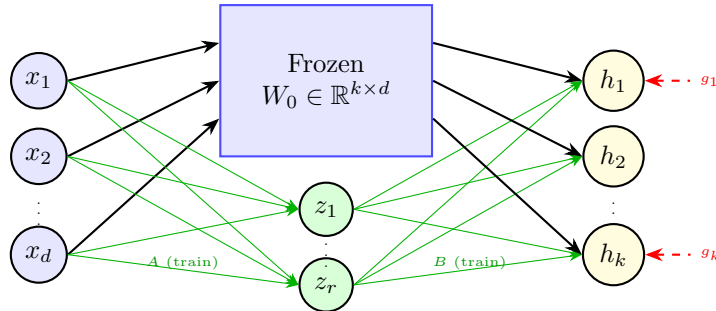


Figure 11: LoRA: input flows through frozen W_0 and trainable low-rank path $A \rightarrow B$; outputs are summed.

State Transition Table

Property	Before	After
Base weights W_0	requires_grad = True	Frozen (False)
Trainable params	$k \times d$	$r(k + d) \ll kd$
Matrix B	Does not exist	$\mathbb{R}^{k \times r}$, init = 0
Matrix A	Does not exist	$\mathbb{R}^{r \times d}$, init $\sim \mathcal{N}$
Inference overhead	None	Extra BA multiply (or merged)

Table 11: State properties before and after LoRA injection.

Quantized LoRA (QLoRA)

Mathematical Foundation

QLoRA quantizes the base weight W_0 to 4-bit NormalFloat (NF4) and uses Double Quantization (DQ) to minimize memory requirements. NF4 divides the standard normal distribution $\mathcal{N}(0, 1)$ into $2^k = 16$ equal-area intervals. The quantization levels q_i are calculated using the standard normal quantile function Q_X :

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k} \right) + Q_X \left(\frac{i+1}{2^k} \right) \right) \quad (65)$$

The base weights are normalized to $[-1, 1]$ using block-wise scaling. Let the block size be $B = 64$. For a block W_b , the scale is $c_b = \max(|W_b|)$. The quantized values are:

$$q_t = \arg \min_{i \in \{0, \dots, 15\}} \left| \frac{W_{b,t}}{c_b} - q_i \right| \quad (66)$$

Double Quantization (DQ) quantizes the scaling factors c_b themselves to 8-bit floats (c_b^{FP8}) with a second scale block size of 256, reducing scale memory overhead from $32/64 = 0.5$ bits/weight to $8/64 + 32/(64 \times 256) \approx 0.127$ bits/weight. During backpropagation, the NF4 weights are dequantized to FP16/BF16 to compute outputs, and gradients are computed only for the low-rank parameters:

$$h = \text{dequant}(c_b, W^{\text{NF4}})x + \frac{\alpha}{r} B A x \quad (67)$$

Architecture Flow Diagram

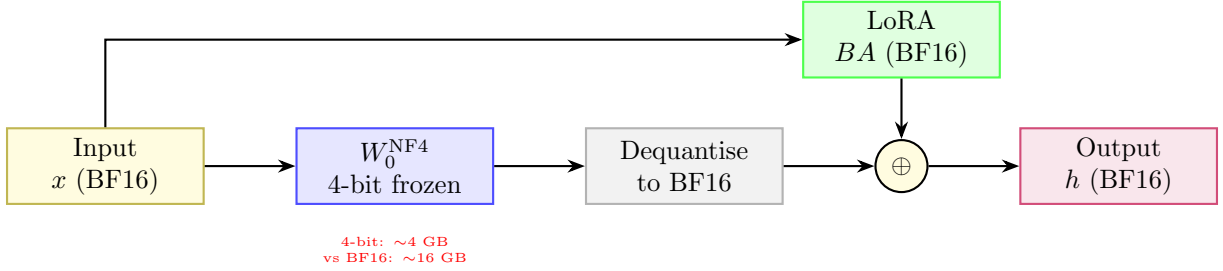


Figure 12: QLoRA: 4-bit frozen base dequantised at runtime; LoRA adapters trained in BF16.

State Transition Table

Property	Before	After
W_0 precision	BF16 / FP16	NF4 (4-bit)
GPU memory for 70B model	~ 140 GB	~ 35 GB
LoRA adapter precision	N/A	BF16
Training fidelity	Full	Near-full (straight-through)
Inference latency	Baseline	+dequant overhead

Table 12: State properties before and after QLoRA.

Adaptive LoRA (AdaLoRA)

Mathematical Foundation

AdaLoRA parameterizes incremental weight updates using a singular value decomposition (SVD) representation to dynamically adjust the adapter rank during training:

$$\Delta W = P\Lambda Q \quad (68)$$

where $P \in \mathbb{R}^{d \times r}$ and $Q \in \mathbb{R}^{r \times k}$ are left and right singular vectors, and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r) \in \mathbb{R}^{r \times r}$ contains singular values. To prevent SVD divergence and maintain parameter stability, we enforce orthogonality constraints:

$$\mathcal{L}_{\text{orth}}(P, Q) = \gamma (\|P^\top P - I\|_F^2 + \|QQ^\top - I\|_F^2) \quad (69)$$

We estimate the training importance of each singular triplet $(P_{*,i}, \lambda_i, Q_{i,*})$ using gradient-weight product magnitude:

$$S_i = s(\lambda_i) + \frac{1}{d} \sum_{j=1}^d s(P_{j,i}) + \frac{1}{k} \sum_{j=1}^k s(Q_{i,j}) \quad (70)$$

where $s(\theta) = |\theta \cdot \nabla_\theta \mathcal{L}|$. At designated intervals, triplets with the lowest importance scores S_i are pruned by zeroing out their singular values λ_i , dynamically reducing the adapter rank budget.

Architecture Flow Diagram

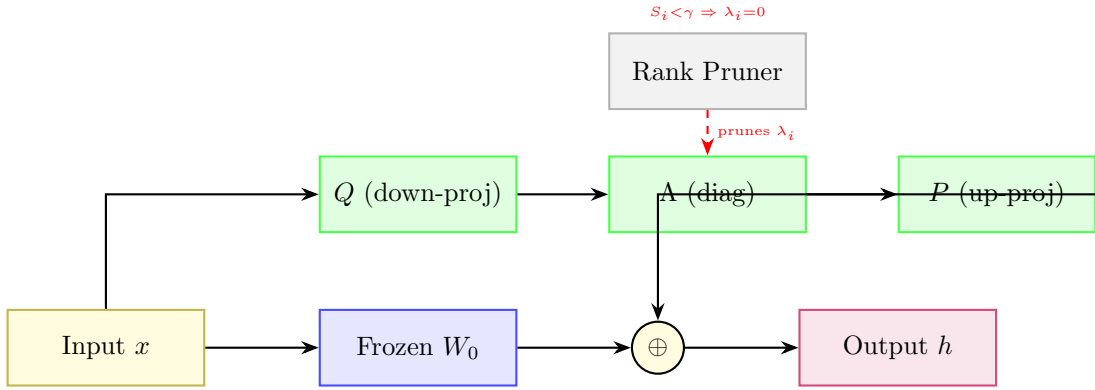


Figure 13: AdaLoRA: SVD-decomposed update with importance-based rank pruning per layer.

State Transition Table

Property	Before	After
Rank per layer	Fixed r	Adaptive (varies by importance)
Update structure	BA product	PAQ SVD triplet
Parameter budget	Fixed	Redistributed across layers
Low-importance ranks	Included	Pruned ($\lambda_i = 0$)
Training stability	Standard	Constrained by SVD orthogonality

Table 13: State properties before and after AdaLoRA.

Weight-Decomposed LoRA (DoRA)

Mathematical Foundation

DoRA decomposes the weight matrix $W \in \mathbb{R}^{k \times d}$ into its magnitude vector $m \in \mathbb{R}^k$ and direction matrix $V \in \mathbb{R}^{k \times d}$:

$$W = m \cdot \frac{V}{\|V\|_r} = m \cdot \frac{W_0 + \Delta W}{\|W_0 + \Delta W\|_r} \quad (71)$$

where $\|\cdot\|_r$ is the L2 norm computed along each row (corresponding to output channels). In parameter-efficient training, we freeze the direction matrix to the base weight $V = W_0$ and introduce low-rank directional updates using adapters $\Delta W = \frac{\alpha}{r} BA$ (where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$):

$$W = m \cdot \frac{W_0 + \frac{\alpha}{r} BA}{\|W_0 + \frac{\alpha}{r} BA\|_r} \quad (72)$$

Only the magnitude vector m and low-rank directional matrices B and A are trainable. The gradient flow for the directional components is:

$$\nabla_V \mathcal{L} = \frac{m}{\|V\|_c} \left(\nabla_W \mathcal{L} - \frac{V}{\|V\|_c^2} \odot (V^\top \nabla_W \mathcal{L}) \right) \quad (73)$$

This mathematically separates the updates to magnitude and direction, mimicking full parameter fine-tuning behavior while retaining low parameter overhead.

Architecture Flow Diagram

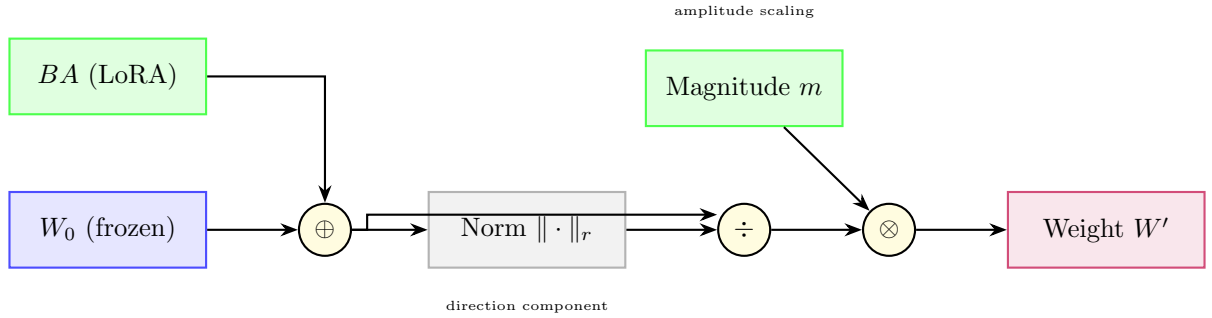


Figure 14: DoRA: weight decomposed into direction (LoRA-adapted) and magnitude (trainable scalar).

State Transition Table

Property	Before (LoRA)	After (DoRA)
Magnitude component	Implicit in W_0	Explicit trainable m
Direction component	$W_0 + BA$ unnorm	Column-normalised
Full FT similarity	Lower	Higher
Trainable params	$r(d + k)$	$r(d + k) + k$
Catastrophic forgetting	Low	Lower

Table 14: State properties comparing LoRA and DoRA.

IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations)

Mathematical Foundation

IA³ introduces element-wise scaling vectors to directly modify inner activations in attention keys, values, and intermediate feed-forward layers. Let the scaling vectors be $l_K \in \mathbb{R}^d$, $l_V \in \mathbb{R}^d$, and $l_{FFN} \in \mathbb{R}^{d_{FFN}}$. The attention layer activations are computed as:

$$\tilde{K} = K \odot l_K = K \cdot \text{diag}(l_K) \quad (74)$$

$$\tilde{V} = V \odot l_V = V \cdot \text{diag}(l_V) \quad (75)$$

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q\tilde{K}^\top}{\sqrt{d_k}}\right)\tilde{V} \quad (76)$$

In the feed-forward network (FFN), let W_1 and W_2 be the weight matrices. The intermediate activations are scaled before the output projection:

$$\text{FFN}(x) = \sigma(xW_1 \odot l_{FFN})W_2 = \sigma(xW_1 \cdot \text{diag}(l_{FFN}))W_2 \quad (77)$$

The base model weights W_Q, W_K, W_V, W_1, W_2 are frozen. This diagonal matrix transformation scales the columns of the projection matrices with minimal parameter updates.

Architecture Flow Diagram

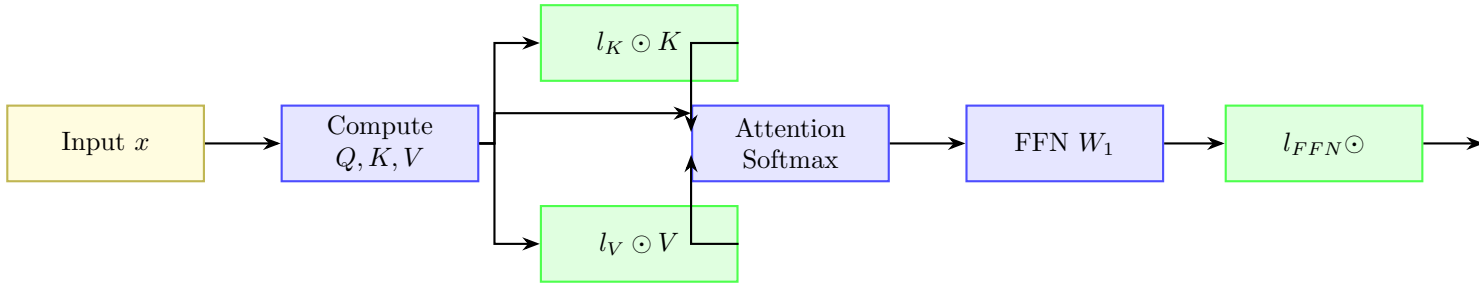


Figure 15: IA³: three tiny scaling vectors rescale keys, values, and FFN intermediate activations.

State Transition Table

Property	Before	After
Trainable params	0 (frozen)	$3dL$ scalars
Init value of l vectors	N/A	1 (identity behaviour)
K, V, FFN activations	Unchanged	Element-wise scaled
Param overhead vs LoRA	N/A	$\sim 10\times$ fewer params
Task adaptation strength	None	Moderate

Table 15: State properties before and after IA³ injection.

Adapter Tuning

Mathematical Foundation

Adapter tuning inserts small bottleneck layers into the transformer block, typically after attention and feed-forward modules. Let $h \in \mathbb{R}^d$ be the input hidden state. In the Pfeiffer architecture, a single adapter is inserted per block. In the Houlsby architecture, two adapters are inserted per block. The mathematical transformation for an adapter block is:

$$h' = h + \sigma(hW_{down})W_{up} \quad (78)$$

where $W_{down} \in \mathbb{R}^{d \times r}$ projects to a bottleneck dimension $r \ll d$, σ is an activation function (typically GELU), and $W_{up} \in \mathbb{R}^{r \times d}$ projects back to the original dimension. To ensure the adapter behaves as an identity mapping at the start of training, W_{up} is initialized to zero:

$$h'|_{t=0} = h + \sigma(hW_{down}) \cdot 0 = h \quad (79)$$

Architecture Flow Diagram

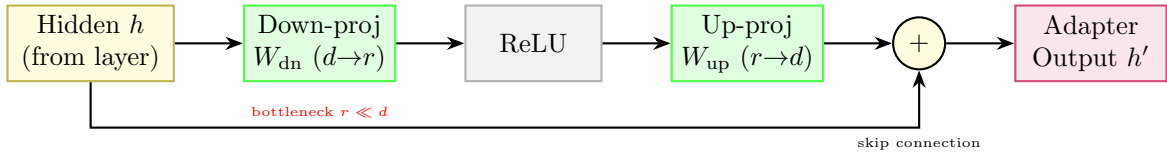


Figure 16: Adapter module: down-project to rank r , apply nonlinearity, up-project back, plus skip connection.

State Transition Table

Property	Before	After
Sub-layer output path	Direct pass-through	+adapter bottleneck
Transformer weights	Trainable	Frozen
New params per layer	0	$2dr + 2d$
Adapter init	N/A	Near-identity ($W_{dn} \sim \mathcal{N}$)
Inference latency	Baseline	+two extra matmuls per layer

Table 16: State properties before and after adapter insertion.

Prefix Tuning

Mathematical Foundation

Prefix Tuning prepends continuous task-specific virtual prefix vectors $P_K, P_V \in \mathbb{R}^{l \times d}$ directly to the Key and Value matrices at every transformer layer. Let the original Key and Value matrices for sequence length T be $K, V \in \mathbb{R}^{T \times d}$. The modified Key and Value matrices become:

$$\tilde{K} = [P_K; K] \in \mathbb{R}^{(l+T) \times d} \quad (80)$$

$$\tilde{V} = [P_V; V] \in \mathbb{R}^{(l+T) \times d} \quad (81)$$

To stabilize optimization, during training the prefix is reparameterized using a Multilayer Perceptron (MLP) over a small source embedding E_k :

$$P_K = \text{MLP}_K(E_k), \quad P_V = \text{MLP}_V(E_k) \quad (82)$$

where $E_k \in \mathbb{R}^{l \times d_{\text{mid}}}$. The MLP is discarded at inference, and only the generated static prefix matrices P_K, P_V are saved and used. The attention output is computed as:

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q[P_K; K]^\top}{\sqrt{d_k}}\right) [P_V; V] \quad (83)$$

Architecture Flow Diagram

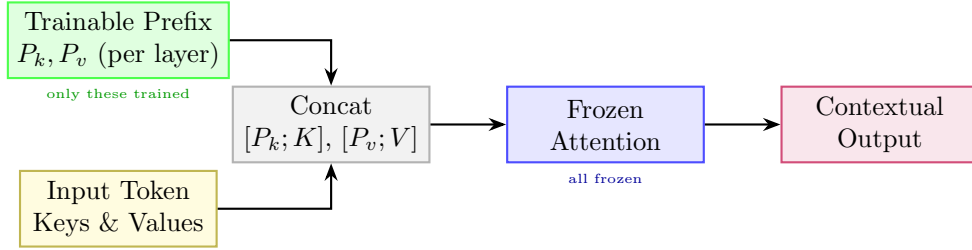


Figure 17: Prefix Tuning: trainable prefix vectors extend the key/value context at every attention layer.

State Transition Table

Property	Before	After
Attention context length	T tokens	$T + l$ tokens (prefix prepended)
Transformer params	Trainable	Fully frozen
Trainable params	0	$2 \times l \times d_h \times L$
Prefix init	N/A	Via MLP reparameterisation
Task switch at inference	Reload model	Swap prefix only

Table 17: State properties before and after prefix tuning.

Prompt Tuning

Mathematical Foundation

Prompt tuning prepends p learnable embedding vectors $P \in \mathbb{R}^{p \times d_{emb}}$ to the input word representations:

$$\tilde{X} = [P; X_e] \in \mathbb{R}^{(p+T) \times d_{emb}} \quad (84)$$

where $X_e \in \mathbb{R}^{T \times d_{emb}}$ is the original sequence word embedding matrix. The entire transformer backbone parameters θ_{base} remain frozen, and gradients are backpropagated only to the prompt parameters P :

$$\nabla_P \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \tilde{X}_{1..p}} \quad (85)$$

During backpropagation, the gradients are passed through all layers of the frozen transformer and update only the prompt embeddings.

Architecture Flow Diagram

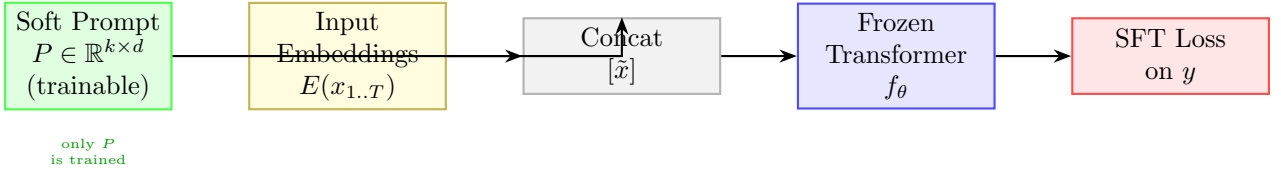


Figure 18: Prompt Tuning: soft tokens prepended at the embedding layer; the full transformer is frozen.

State Transition Table

Property	Before	After
Trainable params	Full model	$k \times d_{emb}$ only
Prompt format	Discrete hand-crafted	Continuous soft vectors
Transformer weights	All trainable	All frozen
Task switching	Reload fine-tuned model	Swap P only
Sensitivity to init	N/A	Strong (init from class labels helps)

Table 18: State properties before and after prompt tuning.

P-Tuning v2

Mathematical Foundation

P-Tuning v2 extends prompt tuning by prepending learnable virtual tokens $P^{(l)} \in \mathbb{R}^{p \times 2d_{\text{attn}}}$ directly to the attention key and value states at every layer $l \in [1, L]$ of the network. Unlike prefix tuning, P-Tuning v2 removes the MLP reparameterization network, directly optimizing the prefix parameters $P^{(l)}$:

$$\tilde{K}^{(l)} = [P_K^{(l)}; K^{(l)}] \quad (86)$$

$$\tilde{V}^{(l)} = [P_V^{(l)}; V^{(l)}] \quad (87)$$

This deep prefix injection resolves the capacity limitation of shallow prompt tuning, allowing small models ($< 10\text{B}$ parameters) to achieve performance comparable to full fine-tuning.

Architecture Flow Diagram

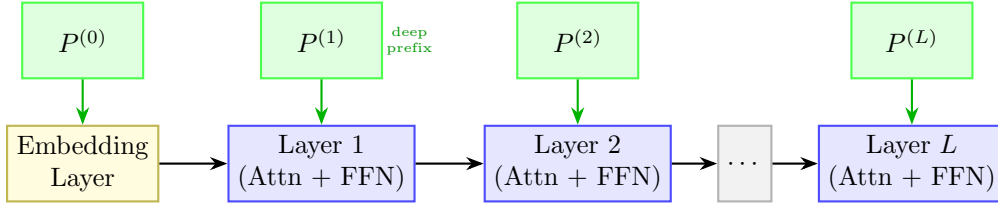


Figure 19: P-Tuning v2: independent trainable prefix tokens injected at every transformer layer.

State Transition Table

Property	Before (Prompt Tuning)	After (P-Tuning v2)
Prefix layers	Embedding only	All L layers
Trainable params	$k \times d$	$L \times l \times d$
NLU performance	Degrades at small scale	Competitive at all scales
Conditioning depth	Shallow (first layer)	Deep (every layer)
Task-specific storage	Single P matrix	L prefix matrices

Table 19: State properties comparing Prompt Tuning and P-Tuning v2.